

[← Go Back](#)

Hardware

Portenta H7 Lite Connected ▾

Tutorials

[BLE Connectivity on Portenta H7](#)[Dual Core Processing](#)[Creating a Flash-Optimized Key-Value Store](#)[Getting Started with OpenMV and MicroPython](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)[Over-The-Air \(OTA\) Updates with the Arduino Portenta H7](#)

Reading and Writing Flash Memory

[Secure Boot on Portenta H7](#)[Setting Up Portenta H7 For Arduino](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Portenta H7 as a USB Host](#)[Portenta H7 as a Wi-Fi Access Point](#)

Home / Hardware / Portenta H7 Lite
Connected / **Reading and Writing Flash Memory**

Reading and Writing Flash Memory

This tutorial demonstrates how to use the on-board Flash memory of the Portenta H7 to read and write data using the BlockDevice API provided by Mbed OS.

	Giampaolo Mancini, Sebastian Romero	Last revision	· 11/03/2022
Author			

ON THIS PAGE

Overview

Goals

Required Hardware and Software

Mbed OS APIs for Flash Storage

Block Device Blocks

Programming the Internal Flash

1. Create the Structure of the Program
2. The Helper Functions
3. Reading & Writing Data
4. Upload the Sketch
5. Results

Programming the QSPI Flash

Conclusion

Next Steps

Overview

This tutorial demonstrates how to use the on-board Flash memory of the Portenta H7 to read and write data using the BlockDevice API provided by Mbed OS. As the internal memory is limited in size, we will also take a look at saving data to the QSPI Flash memory.

Goals

- Accessing the Portenta's internal Flash memory using Mbed's Flash In-Application Programming Interface

- Accessing the Portenta's QSPI Flash memory using Mbed's Flash In-Application Programming Interface

- Reading the memory's characteristics

Required Hardware and Software

- [Portenta H7 \(ABX00042\)](#) or [Portenta H7 Lite Connected \(ABX00046\)](#)

- USB-C® cable (either USB-A to USB-C® or USB-C® to USB-C®)

- Arduino IDE 1.8.10+ or Arduino Pro IDE 0.0.4+ or Arduino CLI 0.13.0+

Mbed OS APIs for Flash Storage

Portenta's core is based on the Mbed operating system, allowing for

Mbed OS has a rich API for managing storage on different mediums, ranging from the small internal Flash memory of a microcontroller to external SecureDigital cards with large data storage space.

In this tutorial, you are going to save a value persistently inside the Flash memory. That allows to access that value even after rebooting the board. You will retrieve some information from a Flash block by using the [FlashIAPBlockDevice](#) API and create a block device object within the available space of the memory. In case of the internal memory, it corresponds to the space which is left after uploading a sketch to the board.

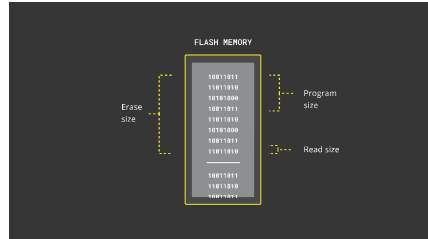
Be aware of the Flash r/w limits: Flash memories have a limited amount of read/write cycles. Typical Flash memories can perform about 10000 writes cycles to the same block before starting to "wear out" and begin to lose the ability to retain data. You can render your board useless with improper use of this example and described APIs.



Block Device Blocks

Blocks of Flash memory can be accessed through the block device APIs. They are byte addressable but operate in units of blocks. There are three types of blocks for the different block device operations: read blocks, erase blocks and program blocks. The recommended

programming it in units of the program block size. The sizes of the erase, program and read blocks may not be the same, but they must be multiples of each another. Keep in mind that the state of an erased block is undefined until you program it with data.



Programming the Internal Flash

1. Create the Structure of the Program

Before we start it's important to keep the above mentioned **Flash r/w limits** in mind! Therefore this method should only be used for **once-in-a-while** read and write **operations**, such as reading a user setting in the `setup()`. It is not a good idea to use it for constantly updated values such as sensor data.

Having this in mind, it is time to create a sketch to program the Portenta. After creating new sketch and giving it a fitting name (in this case `FlashStorage.ino`), you need to create one more file to be used by the sketch, called `FlashIAPLimits.h`, that you will use to define some helper functions. This allows you to reuse the helper file later for other sketches.

2. The Helper Functions

Within the `FlashIAPLimits.h` file, you can start by including necessary libraries and defining the namespace.

```
1 // Ensures that this file
2 #pragma once
3
4 #include <Arduino.h>
5 #include <FlashIAP.h>
6 #include <FlashIAPBlockDe
7
8 using namespace mbed;
```

After that, you can create a struct which will later be used to save the storage's properties.

```
1 // An helper struct for F
2 struct FlashIAPLimits {
3     size_t flash_size;
4     uint32_t start_address;
5     uint32_t available_size
6 };
```

The last part of the helper file consists of the `getFlashIAPLimits()` function used to calculate both the size of the Flash memory as well as the size and start address of the available memory.

This is done with Mbed's **FlashIAP** API. It finds the address of the first sector after the sketch stored in the microcontroller's ROM:

`FLASHIAP_APP_ROM_END_ADDR` and uses the FlashIAP to calculate the Flash memory's size with

other parameters can be determined using the same API.

```

1 // Get the actual start
2 // considering the spac
3 FlashIAPLimits getFlash
4 {
5     // Alignment lambdas
6     auto align_down = [](
7         return ((val) / si
8     };
9     auto align_up = [](ui
10        return ((val - 1)
11    };
12
13     size_t flash_size;
14     uint32_t flash_start_
15     uint32_t start_address
16     FlashIAP flash;
17
18     auto result = flash.i
19     if (result != 0)
20         return { };
21
22     // Find the start of
23     int sector_size = fla
24     start_address = align
25     flash_start_address =
26     flash_size = flash.ge
27
28     result = flash.deinit

```

3. Reading & Writing Data

Going back to the

`FlashStorage.ino` file, some more

files need to be included in order to implement reading and writing to the Flash. The

`FlashIAPBlockDevice.h` API will be

used to create a block device in the empty part of the memory.

Additionally, you can include the

helper file `FlashIAPLimits.h` to

have access to the address and size calculation function that you just created. You can also reference the

```
1 #include <FlashIAPBlockDe
2 #include "FlashIAPLimits.
3
4 using namespace mbed;
```

The `setup()` function will first wait until a serial connection is established and then feed the random number generator, which is used later in this tutorial to write a random number in the Flash memory every time the device boots up.

```
1 void setup() {
2     Serial.begin(115200);
3     while (!Serial);
4
5     Serial.println("FlashIA
6     Serial.println("-----
7
8     // Feed the random numb
9     randomSeed(analogRead(0
```

Next the helper function, defined in the `FlashIAPLimits.h` file is called to calculate the memory properties, which are then used to create a block device using the `FlashIAPBlockDevice.h` library.

```
1 // Get limits of the the
2 auto [flashSize, startAd
3
4 Serial.print("Flash Size
5 Serial.print(flashSize /
6 Serial.println(" MB");
7 Serial.print("FlashIAP S
8 Serial.println(startAddr
9 Serial.print("FlashIAP S
10 Serial.print(iapSize / 1
11 Serial.println(" MB");
12
13 // Create a block device
14 FlashIAPBlockDevice bloc
```

Before using the block device, the first step is to initialize it using `blockDevice.init()`. Once initialized, it can provide the sizes of the blocks for programming the Flash. In terms of reading and writing Flash memory blocks, there is a distinction between the size of a readable block in bytes, a programmable block, which size is always a multiple of the read size, and an erasable block, which is always a multiple of a programmable block.

When reading and writing directly from and to the Flash memory, you need to always allocate a buffer with a multiple of the program block size. The amount of required program blocks can be determined by dividing the data size by the program block size. The final buffer size is equal to the amount of program blocks multiplied by the program block size.


```
1 // Initialize the Flash
2 blockDevice.init();
3
4 const auto eraseBlockSize = 1024;
5 const auto programBlockSize = 1024;
6
7 Serial.println("Block device initialized");
8 Serial.println("Readable");
9 Serial.println("Programmable");
10 Serial.println("Erasable");
11
12 String newMessage = "Random message";
13
14 // Calculate the amount of data to write
15 // This has to be a multiple of the program block size
16 const auto messageSize = newMessage.length();
17 const unsigned int requiredDataSize = (messageSize / programBlockSize) * programBlockSize;
18 const unsigned int requiredErasedSize = requiredDataSize;
19 const auto dataSize = requiredErasedSize + requiredDataSize;
20 char buffer[dataSize] {}
```

In the last part of the `setup()` function you can now use the block device to read and write data. First the buffer is used to read what was stored within the previous execution, then the memory gets erased and reprogrammed with the new content. At the end of the reading and writing process, the block device needs to be de-initialized again using `blockDevice.deinit()`.

```
1 // Read back what was st
2 Serial.println("Reading
3 blockDevice.read(buffer,
4 Serial.println(buffer);
5
6 // Erase a block startin
7 // to the block device s
8 blockDevice.erase(0, req
9
10 // Write an updated mess
11 Serial.println("Writing
12 Serial.println(newMessag
13 blockDevice.program(newM
14
15 // Deinitialize the devi
16 blockDevice.deinit();
17 Serial.println("Done.");
```

Finally the `loop()` function of this sketch will be left empty, considering that the Flash reading and writing process should be carried out as little as possible.

4. Upload the Sketch

Below is the complete sketch of this tutorial consisting of the main sketch and the `FlashIAPLimits.h` helper file, upload both of them to your Portenta H7 to try it out.

FlashIAPLimits.h

```
1  /**
2  Helper functions for ca
3  **/
4
5  // Ensures that this fi
6  #pragma once
7
8  #include <Arduino.h>
9  #include <FlashIAP.h>
10 #include <FlashIAPBlock
11
12 using namespace mbed;
13
14 // A helper struct for
15 struct FlashIAPLimits {
16     size_t flash_size;
17     uint32_t start_addres
18     uint32_t available_si
19 };
20
21 // Get the actual start
22 // considering the spac
23 FlashIAPLimits getFlash
24 {
25     // Alignment lambdas
26     auto align_down = [](
27         return ((val) / si
28     };
29
```

FlashStorage.ino

```
1 #include <FlashIAPBlockDevice>
2 #include "FlashIAPLimits.h"
3
4 using namespace mbed;
5
6 void setup() {
7     Serial.begin(115200);
8     while (!Serial);
9
10    Serial.println("Flash IAP Block Device");
11    Serial.println("-----");
12
13    // Feed the random number generator
14    randomSeed(analogRead(0));
15
16    // Get limits of the flash
17    auto [flashSize, startAddress, iapSize] =
18        FlashIAPLimits::getLimits();
19    Serial.print("Flash Size: ");
20    Serial.print(flashSize);
21    Serial.println(" MB");
22    Serial.print("Flash IAP Start Address: ");
23    Serial.println(startAddress);
24    Serial.print("Flash IAP Size: ");
25    Serial.print(iapSize);
26    Serial.println(" MB");
27
28    // Create a block device
```

5. Results

After uploading the sketch open the Serial Monitor to start the Flash reading and writing process. The first time you start the script, the block device will be filled randomly. Now try to reset or disconnect the Portenta and reconnect it, you should see a message with the random number written to the Flash storage in the previous execution.

Note that the value written to the Flash storage will persist if the board is reset or disconnected. However, the

i Flash storage will be reprogrammed once a new sketch is uploaded to the Portenta and may overwrite the data stored in the Flash.

Programming the QSPI Flash

One issue with the internal Flash is that it is limited in size and the erase blocks are pretty large. This leaves very little space for your sketch and you may quickly run into issues with more complex applications.

Therefore, you can use the external QSPI Flash which has plenty of space to store data. For that, the block device needs to be initialized differently, but the rest of the sketch remains the same. To initialize the device you can use the [QSPIFBlockDevice](#) class which is a block device driver for NOR-based QSPI Flash devices.

```
1 #define BLOCK_DEVICE_SIZ
2 #define PARTITION_TYPE 0
3
4 // Create a block device
5 QSPIFBlockDevice root(PD
6 MBRBlockDevice blockDevi
7
8 // Initialize the Flash
9 if(blockDevice.init() !=
10     Serial.println("Partit
11     blockDevice.deinit();
12     // Allocate a FAT 32 p
13     MBRBlockDevice::partit
14     blockDevice.init();
```

The full QSPI version of the sketch is as follows:

We have learned how to use the

save custom data. It is not recommended to use the Flash of the microcontroller as the primary storage for data-intensive applications. It is better suited for read/write operations that are performed only once in a while such as storing and retrieving application configurations or persistent parameters.

Next Steps

Now that you know how to use block device to perform reading and writing Flash memory, you can look into the [next tutorial](#) on how to use the [TDBStore API](#) to create a [key value store](#) in the Flash memory.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

